

STARTUP INVESTMENT ANALYSIS AND MANAGEMENT SYSTEM

**UCS310 – Database Management Systems
B.E (2nd Year)
Computer Engineering (COE)**

GROUP MEMBERS (2C24):

**Vanshaj Goel (1024030366)
Ayush Garg (1024030878)
Kashish (1024030371)**



Submitted to: Ms. Amrita

**THAPAR INSTITUTE OF ENGINEERING AND TECHNOLOGY,
PATIALA, PUNJAB, INDIA**

Session-Year (Jan–May, 2026)

INDEX

Sr. No.	Section Title	Page No.
1	Introduction & Motivation	3
1.1	Motivation & Fixing Real Problems	3
1.2	Why a DBMS Matters	3
1.3	Automation with PL/SQL	3
2	Problem Statement & Scope	4
3	Objectives of the Project	4
4	Scope of the Project	4
5	Proposed System Description	5
6	Database Design & ER Diagram	5
6.1	Entities and Their Attributes	5
6.2	Relationships	6
6.3	ER Diagram – Chen Notation	6
7	Relational Schema Tables	7
8	SQL Queries – Overview	9
9	Normalization – In Depth	10
9.1	Unnormalized Form (UNF)	10
9.2	First Normal Form (1NF)	10
9.3	Second Normal Form (2NF)	11
9.4	Third Normal Form (3NF)	11
9.5	Boyce-Codd Normal Form (BCNF)	12
9.6	Normalization Summary	12
10	Database Implementation	13
11	Transaction Management & Concurrency	13
12	Tools & Technologies Used	14
13	Expected Outcomes	14

1. INTRODUCTION & MOTIVATION

The Startup Investment Analysis and Management System is a robust, database-backed application built to address the challenges of venture capital and angel investing. By centralizing investments, startup profiles, and financial records in one place, it supports better data-driven decision-making.

1.1 Motivation & Fixing Real Problems

Most traditional firms rely on manual tools or messy spreadsheets, leading to duplicate entries and risk of losing critical financial data. This system eliminates that—clean links between money, companies, and performance replace fragmented records.

1.2 Why a DBMS Matters

Moving from Excel to a real RDBMS ensures data correctness, supports concurrent access by multiple investors and analysts, enhances security, and enables fast historical data retrieval through SQL.

1.3 Automation with PL/SQL

Using PL/SQL triggers and stored procedures, the system automates equity dilution tracking, ROI calculation, and portfolio value updates—eliminating human calculation errors entirely.

2. PROBLEM STATEMENT & SCOPE

Manual systems suffer from duplicate entries, inaccurate valuation records, and missing links between deals and funding rounds. This project builds a normalized relational database that stores investor information, tracks sectors, logs every transaction correctly, and uses constraints and triggers to maintain data integrity automatically.

3. OBJECTIVES OF THE PROJECT

- Design a normalized database using ER models and convert it to a relational schema.
- Apply 3NF normalization to eliminate redundancy and transitive dependencies.
- Build the backend with SQL DDL and DML for all CRUD operations.
- Implement triggers and stored procedures for automation of business logic.
- Enforce ACID properties to ensure transaction safety and consistency.
- Enable fast portfolio search and real-time updates for all stakeholders.
- Generate analytical reports on sector performance and investor portfolios.
- Support data-driven investment decision-making at all management levels.

4. SCOPE OF THE PROJECT

The project covers five key areas: managing investors, registering startups, recording financial transactions, tracking performance statistics, and analyzing investment risks. Three user roles are supported:

- Admin: Manages profiles and overall data governance.
- Analyst: Conducts research and financial calculations.
- Investor: Views and monitors personal portfolio history.

5. PROPOSED SYSTEM DESCRIPTION

A centralized hub where admins manage profiles and analysts record deals—all on a MySQL backend. When money is invested, triggers fire automatically to update portfolio ownership percentages, ensuring consistency at all times.

Key Features:

- Auto-IDs: Eliminates manual typing and duplicate entries through AUTO_INCREMENT primary keys.
- Dilution Tracking: A trigger monitors equity percentages to ensure total ownership never exceeds 100%.
- Performance Tracking: ROI columns show profit and loss in real time.
- Analytics Views: Fast aggregate reports on sector performance and portfolio health.

6. DATABASE DESIGN & ER DIAGRAM

6.1 Entities and Their Attributes

The following six entities are identified in the system based on the ER Diagram:

Entity	Primary Key	Attributes
SECTOR	SectorID	SectorName
INVESTOR	InvestorID	FullName, InvestorType, Email
STARTUP	StartupID	StartupName, FoundingDate, SectorID (FK)
INVESTMENT	InvestmentID	InvestorID (FK), StartupID (FK), InvestedAmount, EquityPercentage, TransactionDate, FundingRound
FINANCIAL_DATA	FinancialID	StartupID (FK), Revenue, NetProfit, ReportYear, ReportQuarter
RISK_ASSESSMENT	RiskID	InvestmentID (FK), RiskScore, AssessmentDate, Notes

6.2 Relationships

- INVESTOR — Makes (1:N) — INVESTMENT: One investor can make many investments.
- STARTUP — Receives (1:N) — INVESTMENT: One startup can receive many investments.
- SECTOR — Categorizes (1:N) — STARTUP: One sector categorizes many startups.
- STARTUP — Has (1:N) — FINANCIAL_DATA: One startup has many financial records.
- INVESTMENT — Associated With (1:N) — RISK_ASSESSMENT: One investment can have many risk assessments.

6.3 ER Diagram – Chen Notation

Rectangles = Entities, Ovals = Attributes, Diamonds = Relationships, Underlined ovals = Primary Keys, 1/N labels = Cardinality.

Startup Investment Analysis System - Detailed ER Diagram (Chen Notation)

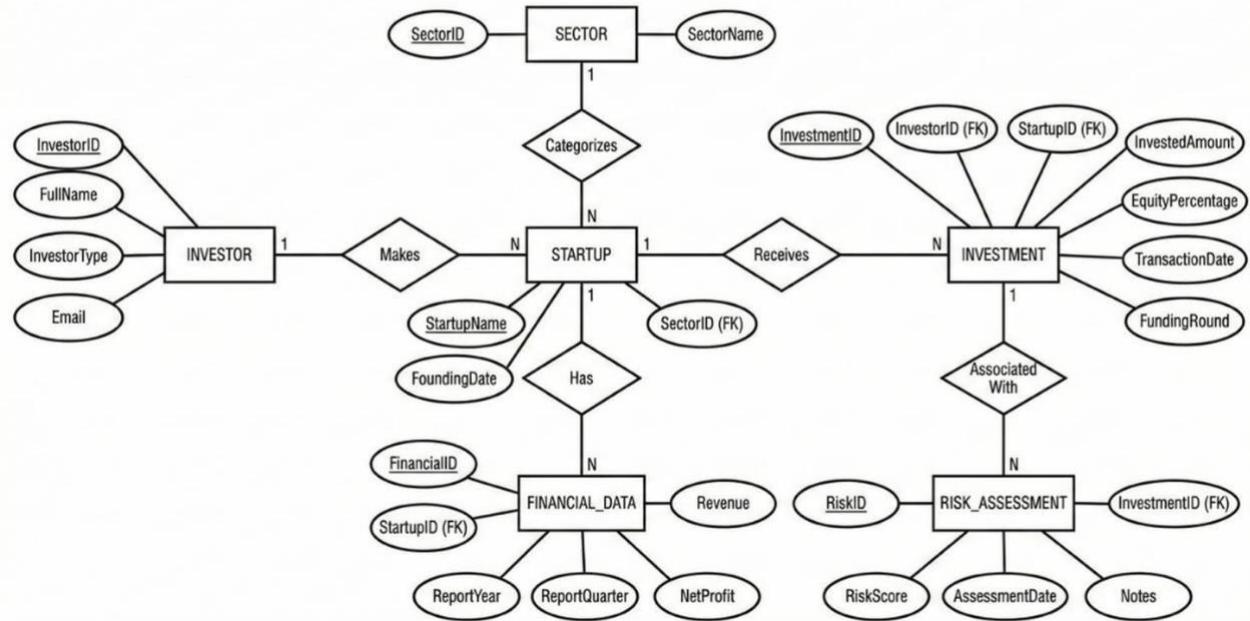


Figure 1: Startup Investment Analysis System – Detailed ER Diagram (Chen Notation)

7. RELATIONAL SCHEMA TABLES

The following six tables are derived from the ER diagram, each with primary keys, foreign keys, and data constraints:

Table 1: SECTOR

Column	Data Type	Constraints	Description
SectorID	INT	PK, AUTO_INCREMENT	Unique sector identifier
SectorName	VARCHAR(100)	NOT NULL, UNIQUE	Industry sector name

Table 2: INVESTOR

Column	Data Type	Constraints	Description
InvestorID	INT	PK, AUTO_INCREMENT	Unique investor identifier
FullName	VARCHAR(150)	NOT NULL	Full name of investor
InvestorType	VARCHAR(50)	NOT NULL	Angel / VC / Institutional

Email	VARCHAR(100)	NOT NULL, UNIQUE	Investor contact email
-------	--------------	------------------	------------------------

Table 3: STARTUP

Column	Data Type	Constraints	Description
StartupID	INT	PK, AUTO_INCREMENT	Unique startup identifier
StartupName	VARCHAR(150)	NOT NULL, UNIQUE	Registered startup name
FoundingDate	DATE	NOT NULL	Date startup was founded
SectorID	INT	FK → SECTOR(SectorID)	Sector the startup belongs to

Table 4: INVESTMENT

Column	Data Type	Constraints	Description
InvestmentID	INT	PK, AUTO_INCREMENT	Unique investment identifier
InvestorID	INT	FK → INVESTOR	Investor who made the deal
StartupID	INT	FK → STARTUP	Startup receiving investment
InvestedAmount	DECIMAL(15,2)	NOT NULL, CHECK > 0	Amount in USD
EquityPercentage	DECIMAL(5,2)	CHECK 0–100	Equity stake acquired
TransactionDate	DATE	NOT NULL	Date of investment
FundingRound	VARCHAR(50)	NOT NULL	Seed / Series A / B / C

Table 5: FINANCIAL_DATA

Column	Data Type	Constraints	Description
FinancialID	INT	PK, AUTO_INCREMENT	Unique financial record ID
StartupID	INT	FK → STARTUP	Startup this record belongs to
Revenue	DECIMAL(15,2)	NOT NULL	Total revenue for the period
NetProfit	DECIMAL(15,2)	NOT NULL	Net profit or loss
ReportYear	YEAR	NOT NULL	Year of financial report
ReportQuarter	TINYINT	CHECK 1–4	Quarter of the report (1-4)

Table 6: RISK_ASSESSMENT

Column	Data Type	Constraints	Description
RiskID	INT	PK, AUTO_INCREMENT	Unique risk record ID
InvestmentID	INT	FK → INVESTMENT	Associated investment
RiskScore	DECIMAL(4,2)	CHECK 0–10	Risk score (0=low, 10=high)
AssessmentDate	DATE	NOT NULL	Date assessment was done
Notes	TEXT	NULLABLE	Analyst observations

8. SQL QUERIES – OVERVIEW

The system uses four main categories of SQL operations. The following gives a high-level overview of the types of queries used:

8.1 DDL – Data Definition Language

CREATE TABLE statements build all six tables with primary keys, foreign keys, NOT NULL constraints, UNIQUE constraints, and CHECK constraints. ALTER TABLE is used to add or modify constraints after initial creation if required.

8.2 DML – Data Manipulation Language

INSERT INTO statements load investor, startup, and investment records. UPDATE statements modify existing data such as equity percentages or contact information. DELETE statements remove records while referential integrity constraints prevent orphaned data.

8.3 DQL – Analytical SELECT Queries

SELECT queries with JOIN, GROUP BY, HAVING, and ORDER BY clauses answer key business questions: total investment per startup, average risk score per sector, investor portfolio summaries, startups with equity exceeding a threshold, and financial performance trends across quarters.

8.4 Triggers & Stored Procedures

A BEFORE INSERT trigger on the INVESTMENT table validates that total equity for a startup does not exceed 100%, signaling an error if violated. A second trigger auto-populates AssessmentDate in RISK_ASSESSMENT when not provided. A stored procedure calculates ROI for any given investment by comparing invested capital against the startup's net profit records.

9. NORMALIZATION – IN DEPTH

Normalization is a systematic process of organizing database tables to reduce redundancy and improve data integrity by applying a series of rules called Normal Forms (NF). Each form resolves specific types of data anomalies.

9.1 Unnormalized Form (UNF) – The Starting Problem

Before normalization, all data resided in one flat table. This caused three classic anomalies:

- Update Anomaly: Investor name and email repeated for every investment, meaning one change required many updates.
- Deletion Anomaly: Deleting the last investment for a startup would permanently remove all sector and revenue data.
- Insertion Anomaly: A new investor could not be added to the system without also recording an investment.

9.2 First Normal Form (1NF)

Rule: All columns must contain atomic (indivisible) values, no repeating groups, and each row must be uniquely identifiable.

- Revenue was stored once for multiple quarters — separated into individual rows per quarter in FINANCIAL_DATA.
- Multiple risk scores were stored comma-separated — each score is now a separate row in RISK_ASSESSMENT.
- No primary key existed on the flat table — auto-increment IDs added for each entity.

9.3 Second Normal Form (2NF)

Rule: Must be in 1NF, and every non-key attribute must be fully functionally dependent on the entire primary key (no partial dependencies). Partial dependencies occur only in tables with composite primary keys.

Violations fixed — in a hypothetical merged table with composite key (InvestorID + StartupID):

- InvestorName depended only on InvestorID, not on the full key → moved to the standalone INVESTOR table.
- StartupName depended only on StartupID → moved to the standalone STARTUP table.
- INVESTMENT now stores only attributes that depend on InvestmentID alone: InvestedAmount, EquityPercentage, TransactionDate, FundingRound.

9.4 Third Normal Form (3NF)

Rule: Must be in 2NF, and there must be no transitive dependencies (no non-key attribute may depend on another non-key attribute).

- SectorName depended transitively on StartupID via SectorID (StartupID → SectorID → SectorName) — SectorName moved to the separate SECTOR table.
- InvestorEmail was removed from INVESTMENT — all investor attributes live exclusively in the INVESTOR table referenced by FK.

9.5 Boyce-Codd Normal Form (BCNF)

Rule: For every non-trivial functional dependency $X \rightarrow Y$, X must be a superkey. All six tables satisfy BCNF since each has a single simple primary key, every functional dependency has that primary key as the determinant, and no attribute determines another outside its own table scope.

9.6 Normalization Summary

Normal Form	Rule	Problem Solved	Applied In
1NF	Atomic values, no repeating groups	Removed multi-valued Revenue and RiskScore columns	All tables
2NF	No partial dependencies	Separated InvestorName & StartupName from INVESTMENT	INVESTOR, STARTUP, INVESTMENT
3NF	No transitive dependencies	Moved SectorName to SECTOR; removed Email from INVESTMENT	SECTOR, STARTUP, INVESTMENT
BCNF	Every determinant is a superkey	No overlapping candidate keys; all determinants are PKs	All tables

Table	Primary Key	Transitive Dep?	3NF Status
SECTOR	SectorID	None	✓ Compliant
INVESTOR	InvestorID	None	✓ Compliant
STARTUP	StartupID	None	✓ Compliant
INVESTMENT	InvestmentID	None	✓ Compliant
FINANCIAL_DATA	FinancialID	None	✓ Compliant
RISK_ASSESSMENT	RiskID	None	✓ Compliant

10. DATABASE IMPLEMENTATION

The system is built on MySQL with the InnoDB storage engine. DDL constructs the schema while DML manages data. Aggregate functions such as SUM and AVG support portfolio analytics. PRIMARY KEY and FOREIGN KEY constraints enforce referential integrity. Triggers automate equity validation and ROI tracking to eliminate human errors.

11. TRANSACTION MANAGEMENT & CONCURRENCY CONTROL

ACID properties are central to the transaction model of this system:

- Atomicity: Investment recording and portfolio updates are wrapped in a single transaction. If any step fails, ROLLBACK undoes all changes, leaving the database in a consistent state.
- Consistency: CHECK constraints and triggers ensure equity percentages never exceed 100% and all financial figures are non-negative at all times.

- Isolation: InnoDB's row-level locking prevents concurrent updates from interfering. SELECT...FOR UPDATE is used when reading equity totals before insertion to prevent race conditions.
- Durability: Upon COMMIT, all changes are written to disk and survive system crashes via MySQL's write-ahead logging (WAL).

SAVEPOINT usage: A SAVEPOINT is created before equity validation. If the new investment would cause total equity to exceed 100%, the system rolls back to the SAVEPOINT and rejects the deal, ensuring the cap table is never corrupted.

12. TOOLS & TECHNOLOGIES USED

Category	Technology	Purpose
Database Engine	MySQL 8.x (InnoDB)	Primary RDBMS for storage, indexing, and transactions
Query Language	SQL (DDL + DML + DQL)	Schema creation, data manipulation, and analytics
Procedural SQL	PL/SQL – Triggers & Stored Procs	Business rule automation and ROI calculation
Frontend	HTML5, CSS3, JavaScript	User interface for data entry and portfolio views
Backend	PHP / Python / Java	Application logic and database connectivity
Security	Role-Based Access Control (RBAC)	Admin, Analyst, Investor roles with permission tiers

13. EXPECTED OUTCOMES

- Clean, normalized data with zero redundancy across all six core tables.
- Fast SQL-based portfolio searches replacing manual spreadsheet lookups.
- Automated equity dilution and ROI calculations via triggers and stored procedures.
- Better stakeholder reports generated from analytics views and aggregate queries.
- A scalable, ACID-compliant system demonstrating real DBMS advantages over manual methods.

— END OF DOCUMENT —